

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ОТЧЕТ

по Лабораторной работе № 4

**«Запросы на выборку и модификацию данных. Представления. Работа с
индексами»**

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Забродин Максим Александрович

Группа К3240

Направление подготовки 09.03.03 Прикладная информатика

Преподаватели: Говорова Марина Михайловна, Белов Александр Олегович

Санкт-Петербург

2026

Цель работы

Овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

Практическое задание

- Создать запросы и представления на выборку данных к базе данных PostgreSQL (согласно индивидуальному заданию лабораторной работы №2, часть 2 и 3).
- Составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) с использованием подзапросов.
- Изучить графическое представление запросов и просмотреть историю запросов.
- Создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами. Для получения плана запроса использовать команду EXPLAIN.

Выполнение

Запросы к базе данных

Запрос 1

Формулировка: вывести список поставщиков, которые поставляют все товары.

opt_base/postgres@FantiK

```

1 SET search_path TO wholesale;
2
3 SELECT s.supplier_id, s.supplier_name
4 FROM suppliers s
5 JOIN supplies sp ON s.supplier_id = sp.supplier_id
6 JOIN supply_items si ON sp.supply_id = si.supply_id
7 GROUP BY s.supplier_id, s.supplier_name
8 HAVING COUNT(DISTINCT si.product_id) = (SELECT COUNT(*) FROM products);

```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

supplier_id [PK] integer	supplier_name character varying (100)
1	Alpha Supply LLC

Рисунок 1 — Выполнение запроса 1 и результат

Запрос 2

Формулировка: определить поставщика, который поставяет каждый из товаров по самой низкой цене.

opt_base/postgres@FantiK

```

1 SET search_path TO wholesale;
2
3 SELECT p.product_name, s.supplier_name, si.purchase_unit_price
4 FROM supply_items si
5 JOIN supplies sp ON si.supply_id = sp.supply_id
6 JOIN suppliers s ON sp.supplier_id = s.supplier_id
7 JOIN products p ON si.product_id = p.product_id
8 JOIN (
9     SELECT product_id, MIN(purchase_unit_price) AS min_price
10    FROM supply_items
11   GROUP BY product_id
12 ) AS min_prices
13 ON si.product_id = min_prices.product_id
14 AND si.purchase_unit_price = min_prices.min_price;

```

Data Output Messages Notifications

Showing rows: 1 to 6 Page No: 1 of 1

product_name character varying (100)	supplier_name character varying (100)	purchase_unit_price numeric (12,2)
Sugar	Beta Logistics JSC	55.00
Office Paper	Gamma Trade LLC	1200.00
Sugar	Alpha Supply LLC	55.00
Laptop Model A	Beta Logistics JSC	64000.00
Sugar	Beta Logistics JSC	55.00
Office Paper	Gamma Trade LLC	1200.00

Рисунок 2 — Выполнение запроса 2 и результат

Результат запроса содержит 6 строк, а не 3 (по числу товаров), так как минимальную цену по товарам Sugar и Office Paper делят несколько поставок: товар Sugar поставлялся по одинаковой минимальной цене 55.00 как поставщиком Beta Logistics JSC, так и поставщиком Alpha Supply LLC (в разных поставках), а товар Office Paper по минимальной цене 1200.00 был дважды поставлен поставщиком Gamma Trade LLC. Для товара Laptop Model A минимальная цена 64000.00 однозначно соответствует поставщику Beta Logistics JSC.

Запрос 3

Формулировка: вывести названия товаров, цены на которые у всех поставщиков одинаковы.

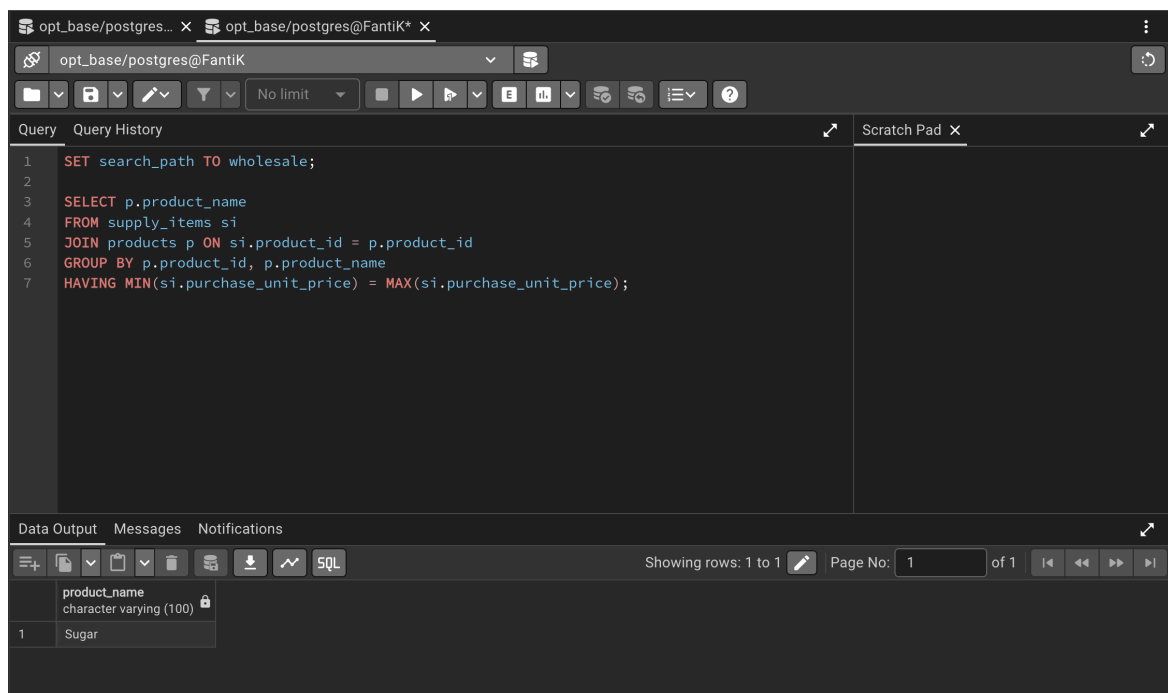


Рисунок 3 — Выполнение запроса 3 и результат

В результат попал только товар Sugar, так как все зафиксированные цены поставки на него равны 55.00. Для товаров Laptop Model A и Office Paper минимальная и максимальная цены поставки не совпадают, поэтому они не входят в результат.

Запрос 4

Формулировка: чему равен общий суточный доход оптового склада за прошедший день?

Примечание: в таблице order_items отсутствует прямая ссылка на заказ (order_id), поэтому дата привязывается к дате поставки соответствующей партии товара (supplies.supply_date) — это единственная доступная в структуре БД дата, связанная с конкретной проданной партией.

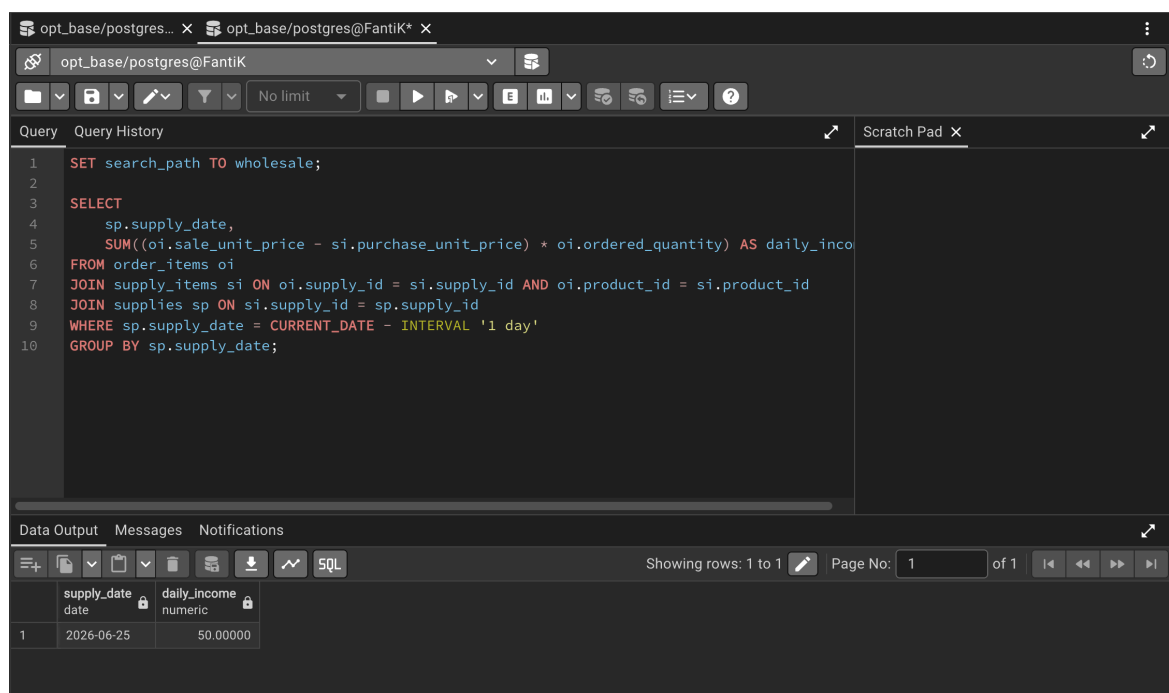


Рисунок 4 — Выполнение запроса 4 и результат

Доход за прошедший день (2026-06-25) составил 50.00 — это разница между ценой продажи (65.00) и ценой закупки (55.00) сахара, умноженная на проданное количество (5 единиц): $(65.00 - 55.00) \times 5 = 50.00$.

Запрос 5

Формулировка: вычислить общую стоимость каждого вида товара, находящегося на базе.

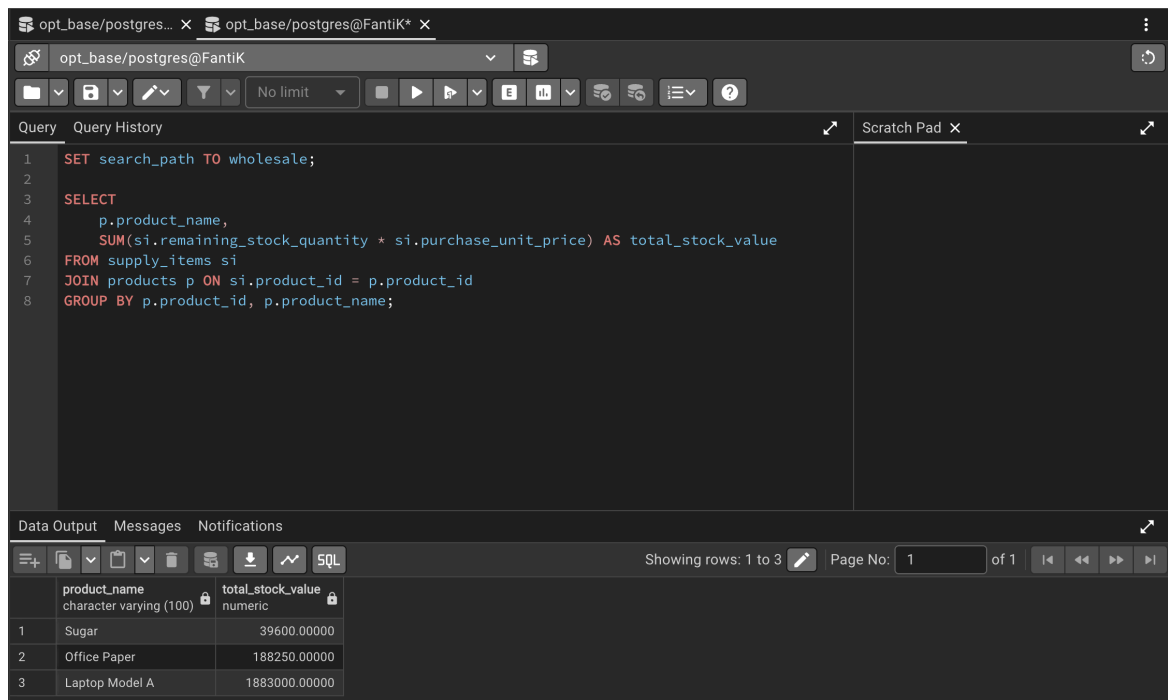


Рисунок 5 — Выполнение запроса 5 и результат

Стоимость остатка товара на складе считается как сумма произведений оставшегося количества (`remaining_stock_quantity`) на закупочную цену по каждой партии поставки данного товара. Наибольшая стоимость остатка — у товара Laptop Model A (1883000.00), наименьшая — у Sugar (39600.00).

Запрос 6

Формулировка: в какой день было вывезено минимальное количество товара?

Примечание: дата привязывается к дате поставки соответствующей партии товара

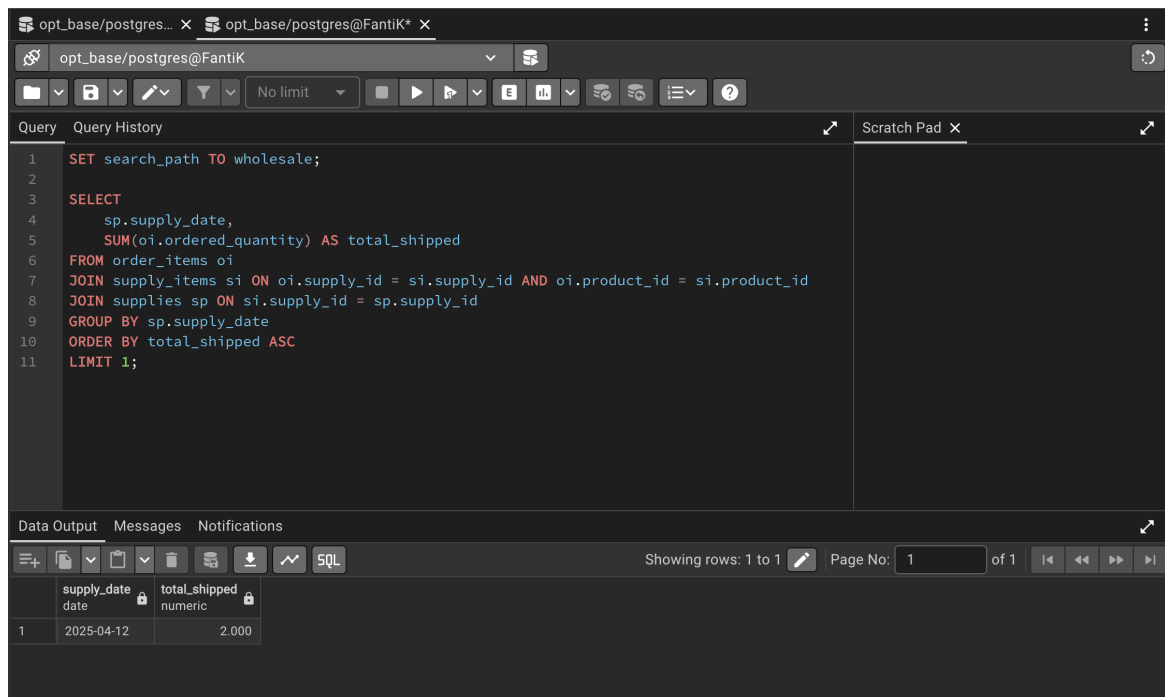


Рисунок 6 — Выполнение запроса 6 и результат

Минимальное суммарное количество вывезенного товара (2 единицы) зафиксировано 12 апреля 2025 года — в этот день было продано 2 ноутбука Laptop Model A из партии, поставленной поставщиком Beta Logistics JSC.

Запрос 7

Формулировка: сколько различных видов товара имеется на базе?

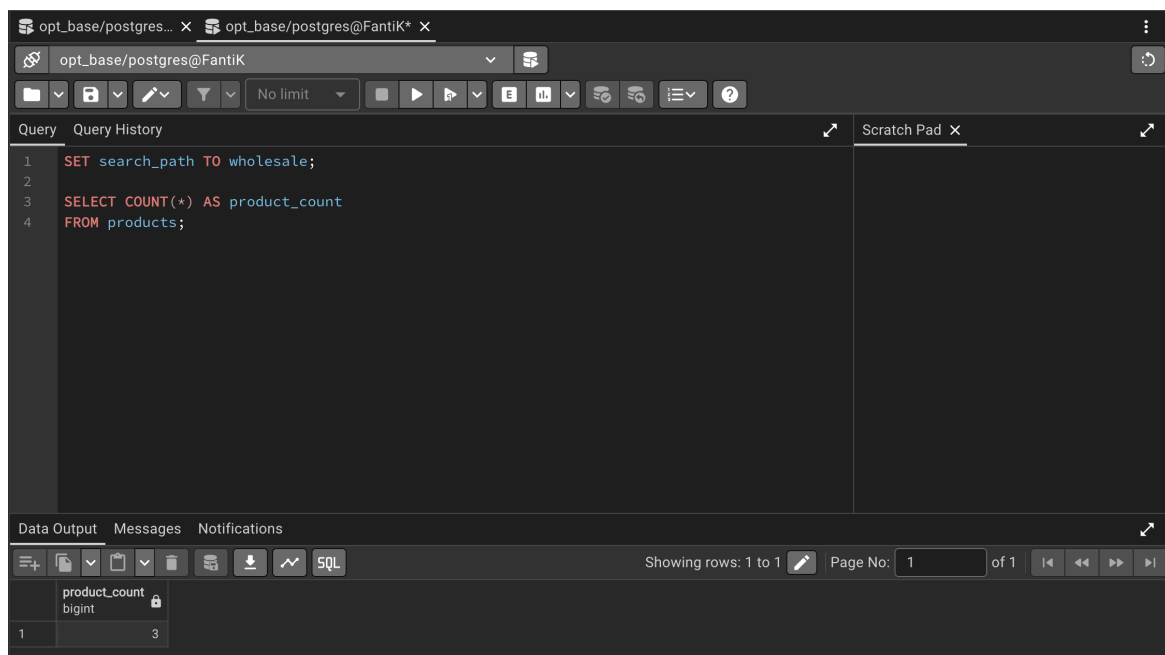


Рисунок 7 — Выполнение запроса 7 и результат

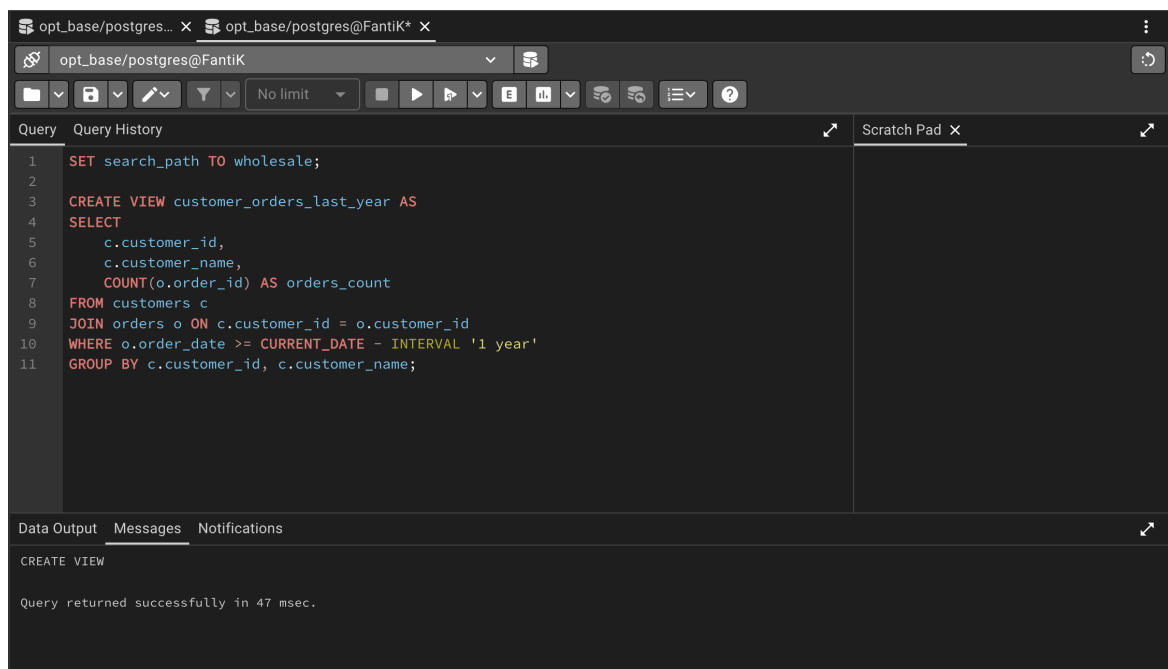
На базе представлено 3 различных вида товара: Laptop Model A, Sugar, Office Paper.

Представления

Представление 1

Формулировка: количество заказов фирм-покупателей за прошедший год.

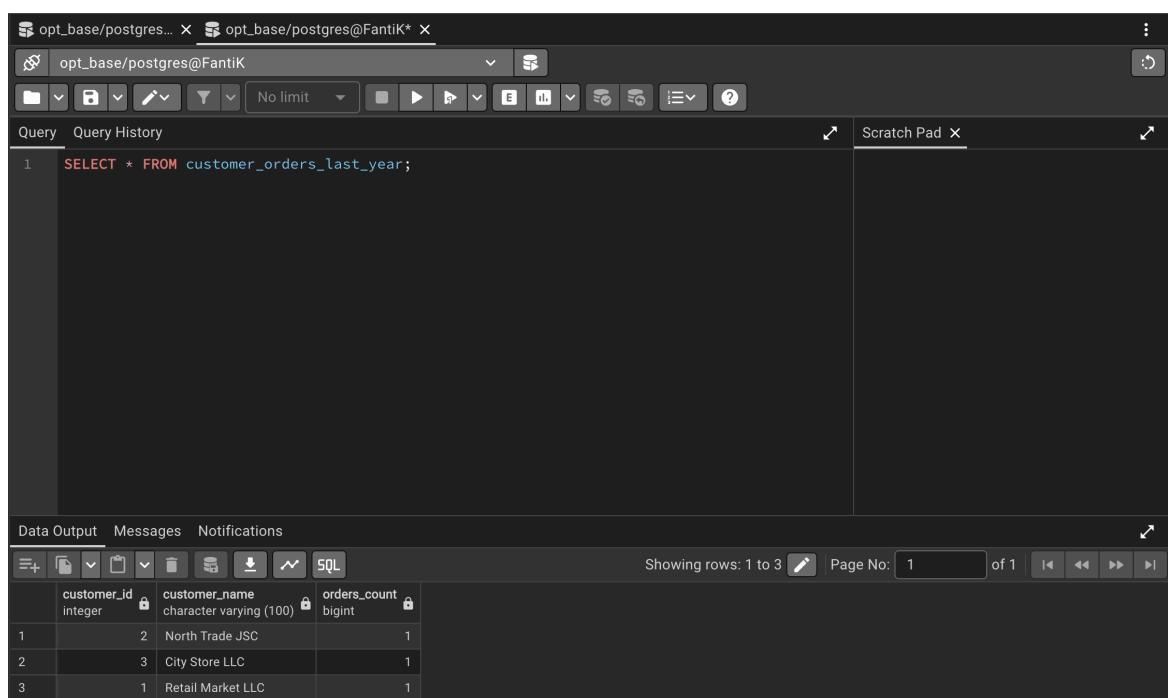
SQL-команда создания представления:



```
1 SET search_path TO wholesale;
2
3 CREATE VIEW customer_orders_last_year AS
4 SELECT
5     c.customer_id,
6     c.customer_name,
7     COUNT(o.order_id) AS orders_count
8 FROM customers c
9 JOIN orders o ON c.customer_id = o.customer_id
10 WHERE o.order_date >= CURRENT_DATE - INTERVAL '1 year'
11 GROUP BY c.customer_id, c.customer_name;
```

Query returned successfully in 47 msec.

Рисунок 8 — Создание представления `customer_orders_last_year`



```
1 SELECT * FROM customer_orders_last_year;
```

	customer_id integer	customer_name character varying (100)	orders_count bigint
1	2	North Trade JSC	1
2	3	City Store LLC	1
3	1	Retail Market LLC	1

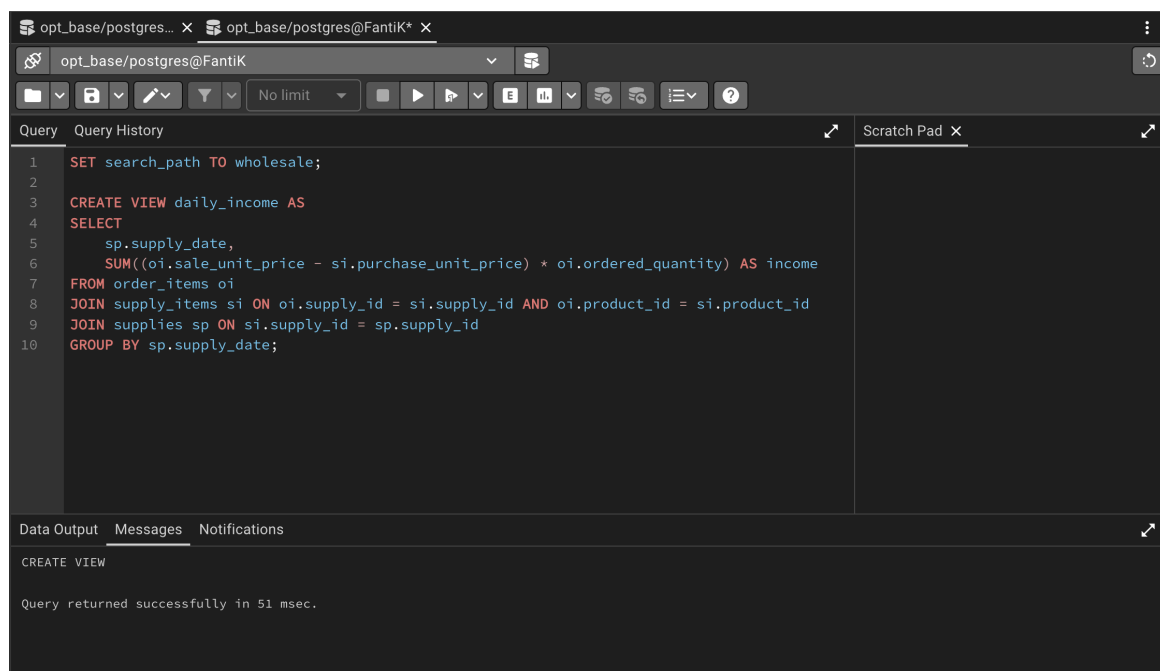
Рисунок 9 — Содержимое представления `customer_orders_last_year`

В представлении отражены 3 покупателя, сделавшие заказы за последний год: North Trade JSC, City Store LLC и Retail Market LLC — по одному заказу у каждого. Покупатели West Trade LLC и South Market JSC не вошли в представление, так как заказов за этот период у них не было.

Представление 2

Формулировка: доход базы за конкретный период.

Примечание: представление выводит доход по каждому дню отдельно (как и в запросе 4 и 6, дата определяется через дату поставки соответствующей партии товара); конкретный период задаётся условием WHERE при обращении к представлению.



```
1 SET search_path TO wholesale;
2
3 CREATE VIEW daily_income AS
4 SELECT
5     sp.supply_date,
6     SUM((oi.sale_unit_price - si.purchase_unit_price) * oi.ordered_quantity) AS income
7 FROM order_items oi
8 JOIN supply_items si ON oi.supply_id = si.supply_id AND oi.product_id = si.product_id
9 JOIN supplies sp ON si.supply_id = sp.supply_id
10 GROUP BY sp.supply_date;
```

Query returned successfully in 51 msec.

Рисунок 10 — Создание представления `daily_income`

Просмотр содержимого представления за период с 1 января 2026 года по текущую дату:

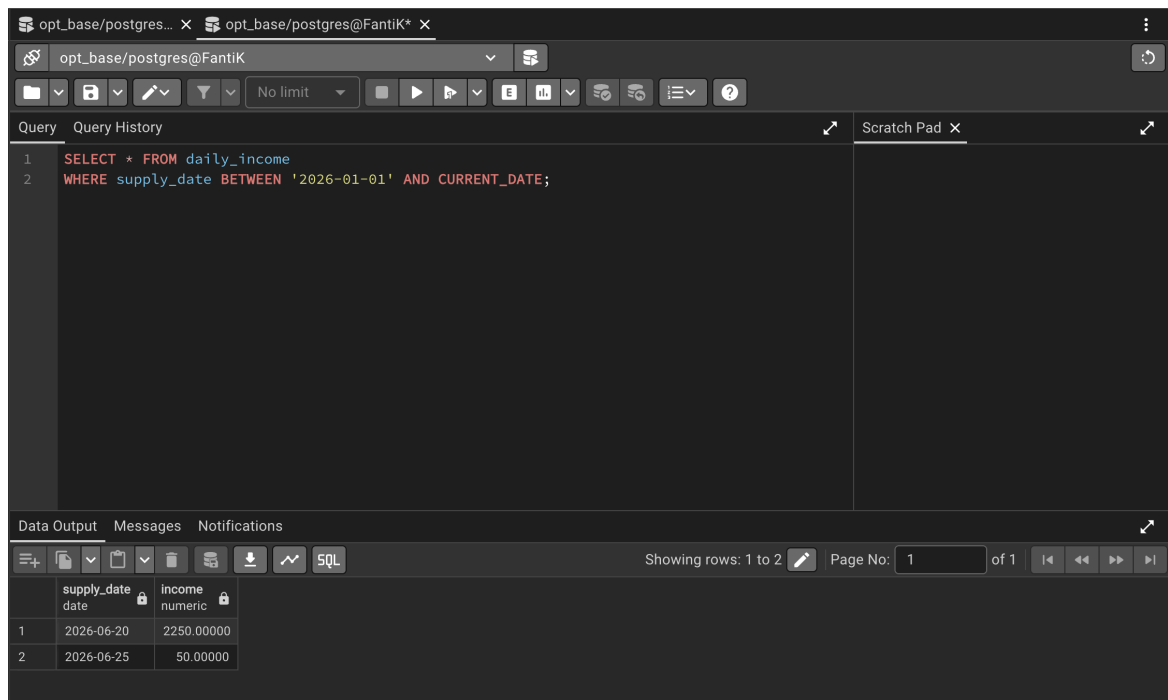


Рисунок 11 — Содержимое представления `daily_income` за заданный период

За указанный период (с 1 января 2026 года по текущую дату) доход зафиксирован за два дня: 2026-06-20 — 2250.00 и 2026-06-25 — 50.00.

Запросы на модификацию данных

В соответствии с заданием составлены и выполнены три запроса на модификацию данных (INSERT, UPDATE, DELETE) с использованием подзапросов.

Запрос на модификацию 1 (INSERT)

Формулировка: добавить новый заказ для покупателя, который сделал наибольшее количество заказов (определяется подзапросом).

Состояние таблицы `orders` до выполнения запроса:

The screenshot shows a PostgreSQL IDE window with the following SQL query in the 'Query' tab:

```

1 SET search_path TO wholesale;
2
3 SELECT * FROM orders ORDER BY order_id;

```

The 'Data Output' tab displays the results of the query in a table format. The table has 5 columns: `order_id` (integer), `order_status` (character varying), `customer_id` (integer), `contract_number` (character varying), and `order_date` (date). The results show 8 rows of data.

order_id	order_status	customer_id	contract_number	order_date
1	Paid	1	SALE-2025-001	2025-03-12
2	Created	2	SALE-2025-001	2025-03-13
3	Assembled	3	MIX-2025-001	2025-03-14
4	Paid	4	SALE-2025-001	2025-04-18
5	Paid	5	SALE-2025-001	2025-05-05
6	Picked up	1	SALE-2025-001	2026-06-20
7	Picked up	2	SALE-2025-001	2026-06-25
8	Picked up	3	SALE-2025-001	2026-06-25

Рисунок 12 — Таблица `orders` до выполнения `INSERT`

The screenshot shows a PostgreSQL IDE window with the following SQL query in the 'Query' tab:

```

1 INSERT INTO orders (order_status, customer_id, contract_number, order_date)
2 VALUES (
3     'Created',
4     (SELECT customer_id FROM orders GROUP BY customer_id ORDER BY COUNT(*) DESC LIMIT 1),
5     'SALE-2025-001',
6     CURRENT_DATE
7 );

```

The 'Data Output' tab shows the result of the query: `INSERT 0 1`. Below this, a message states: `Query returned successfully in 50 msec.`

Рисунок 13 — Выполнение запроса `INSERT` с подзапросом

Состояние таблицы `orders` после выполнения запроса:

opt_base/postgres... x

opt_base/postgres@FantiK* x

opt_base/postgres@FantiK

No limit

<

Рисунок 14 — Таблица `orders` после выполнения `INSERT`

Подзапрос (`SELECT customer_id FROM orders GROUP BY customer_id ORDER BY COUNT(*) DESC LIMIT 1`) определил покупателя с `customer_id = 3` (City Store LLC) как сделавшего наибольшее количество заказов (2 заказа против 1 у остальных). Для него был добавлен новый заказ `order_id = 9` со статусом `Created` и текущей датой.

Запрос на модификацию 2 (UPDATE)

Формулировка: повысить закупочную цену на 10% для товара, у которого остаток на складе (по одной из партий) максимальный (определяется подзапросом).

Состояние таблицы `supply_items` до выполнения запроса:

The screenshot shows a PostgreSQL client window with the following SQL query:

```

1 SET search_path TO wholesale;
2
3 SELECT supply_item_id, supply_id, product_id, purchase_unit_price, remaining_stock_quantity
4 FROM supply_items
5 ORDER BY supply_item_id;

```

The results are displayed in a table with 9 rows:

	supply_item_id [PK] integer	supply_id integer	product_id integer	purchase_unit_price numeric (12,2)	remaining_stock_quantity numeric (12,3)
1	1	1	1	65000.00	15.000
2	2	2	2	55.00	430.000
3	3	3	3	1200.00	80.000
4	4	4	2	55.00	90.000
5	5	5	1	64000.00	8.000
6	6	6	1	66000.00	6.000
7	7	7	3	1250.00	45.000
8	8	8	2	55.00	200.000
9	9	9	3	1200.00	30.000

A status message at the bottom indicates: "Successfully run. Total query runtime: 91 msec. 9 rows affected."

Рисунок 15 — Таблица `supply_items` до выполнения `UPDATE`

The screenshot shows a PostgreSQL client window with the following SQL query:

```

1 UPDATE supply_items
2 SET purchase_unit_price = purchase_unit_price * 1.10
3 WHERE product_id = (
4   SELECT product_id
5   FROM supply_items
6   ORDER BY remaining_stock_quantity DESC
7   LIMIT 1
8 );

```

The status bar at the bottom indicates: "UPDATE 3" and "Query returned successfully in 49 msec."

Рисунок 16 — Выполнение запроса `UPDATE` с подзапросом

Состояние таблицы `supply_items` после выполнения запроса:

opt_base/postgres... x opt_base/postgres@FantiK* x

opt_base/postgres@FantiK

Query Query History Scratch Pad x

```

1 SET search_path TO wholesale;
2
3 SELECT supply_item_id, supply_id, product_id, purchase_unit_price, remaining_stock_quantity
4 FROM supply_items
5 ORDER BY supply_item_id;

```

Data Output Messages Notifications

Showing rows: 1 to 9 Page No: 1 of 1

	supply_item_id [PK] integer	supply_id integer	product_id integer	purchase_unit_price numeric (12,2)	remaining_stock_quantity numeric (12,3)
1	1	1	1	65000.00	15.000
2	2	2	2	60.50	430.000
3	3	3	3	1200.00	80.000
4	4	4	2	60.50	90.000
5	5	5	1	64000.00	8.000
6	6	6	1	66000.00	6.000
7	7	7	3	1250.00	45.000
8	8	8	2	60.50	200.000
9	9	9	3	1200.00	30.000

✓ Successfully run. Total query runtime: 46 msec. 9 rows affected. ✕

Total rows: 9 Query complete 00:00:00.046 15 Lp 5 Col 25

Рисунок 17 — Таблица `supply_items` после выполнения `UPDATE`

Подзапрос определил, что наибольший остаток (430.000) зафиксирован у партии товара с `product_id = 2` (Sugar). Команда `UPDATE` повысила закупочную цену на 10% для всех трёх партий этого товара (`supply_item_id = 2, 4, 8`): цена изменилась с 55.00 на 60.50. Цены на остальные товары (Laptop Model A, Office Paper) не изменились.

Запрос на модификацию 3 (`DELETE`)

Формулировка: удалить самый последний по дате заказ со статусом «Created» (необработанный заказ, ещё не оплаченный и не собранный) — определяется подзапросом.

Состояние таблицы `orders` до выполнения запроса:

The screenshot shows a PostgreSQL IDE interface. The top toolbar includes icons for file operations, query execution, and settings. The 'Query' tab is active, displaying the following SQL commands:

```

1 SET search_path TO wholesale;
2
3 SELECT * FROM orders ORDER BY order_id;

```

The 'Data Output' tab is also active, showing the results of the query. The table has 9 rows and 6 columns: order_id [PK] integer, order_status character varying (30), customer_id integer, contract_number character varying (20), and order_date date. The data is as follows:

order_id [PK] integer	order_status character varying (30)	customer_id integer	contract_number character varying (20)	order_date date
1	Paid	1	SALE-2025-001	2025-03-12
2	Created	2	SALE-2025-001	2025-03-13
3	Assembled	3	MIX-2025-001	2025-03-14
4	Paid	4	SALE-2025-001	2025-04-18
5	Paid	5	SALE-2025-001	2025-05-05
6	Picked up	1	SALE-2025-001	2026-06-20
7	Picked up	2	SALE-2025-001	2026-06-25
8	Picked up	3	SALE-2025-001	2026-06-25
9	Created	3	SALE-2025-001	2026-06-26

Рисунок 18 — Таблица orders до выполнения DELETE

The screenshot shows the same PostgreSQL IDE interface. The 'Query' tab now displays a DELETE statement with a subquery:

```

1 DELETE FROM orders
2 WHERE order_id = (
3   SELECT order_id
4   FROM orders
5   WHERE order_status = 'Created'
6   ORDER BY order_date DESC
7   LIMIT 1
8 );

```

The 'Data Output' tab shows the result of the query: 'DELETE 1'. Below this, a message states: 'Query returned successfully in 58 msec.'

Рисунок 19 — Выполнение запроса DELETE с подзапросом

Состояние таблицы orders после выполнения запроса:

The screenshot shows a PostgreSQL query editor with the following SQL query:

```

1 SET search_path TO wholesale;
2
3 SELECT * FROM orders ORDER BY order_id;

```

The results of the query are displayed in a table with 8 rows and 5 columns:

	order_id [PK] integer	order_status character varying (30)	customer_id integer	contract_number character varying (20)	order_date date
1	1	Paid	1	SALE-2025-001	2025-03-12
2	2	Created	2	SALE-2025-001	2025-03-13
3	3	Assembled	3	MIX-2025-001	2025-03-14
4	4	Paid	4	SALE-2025-001	2025-04-18
5	5	Paid	5	SALE-2025-001	2025-05-05
6	6	Picked up	1	SALE-2025-001	2026-06-20
7	7	Picked up	2	SALE-2025-001	2026-06-25
8	8	Picked up	3	SALE-2025-001	2026-06-25

Рисунок 20 — Таблица orders после выполнения DELETE

Подзапрос определил `order_id = 9` (заказ покупателя City Store LLC от 2026-06-26, ранее добавленный запросом INSERT) как самый последний заказ со статусом Created. Команда DELETE удалила эту запись — после выполнения в таблице orders снова 8 строк, заказ с `order_id = 9` отсутствует.

Графическое представление запроса и история запросов

Для запроса 5 (общая стоимость каждого вида товара на базе) средствами вкладки Explain (режим Graphical) в pgAdmin 4 получен графический план выполнения:

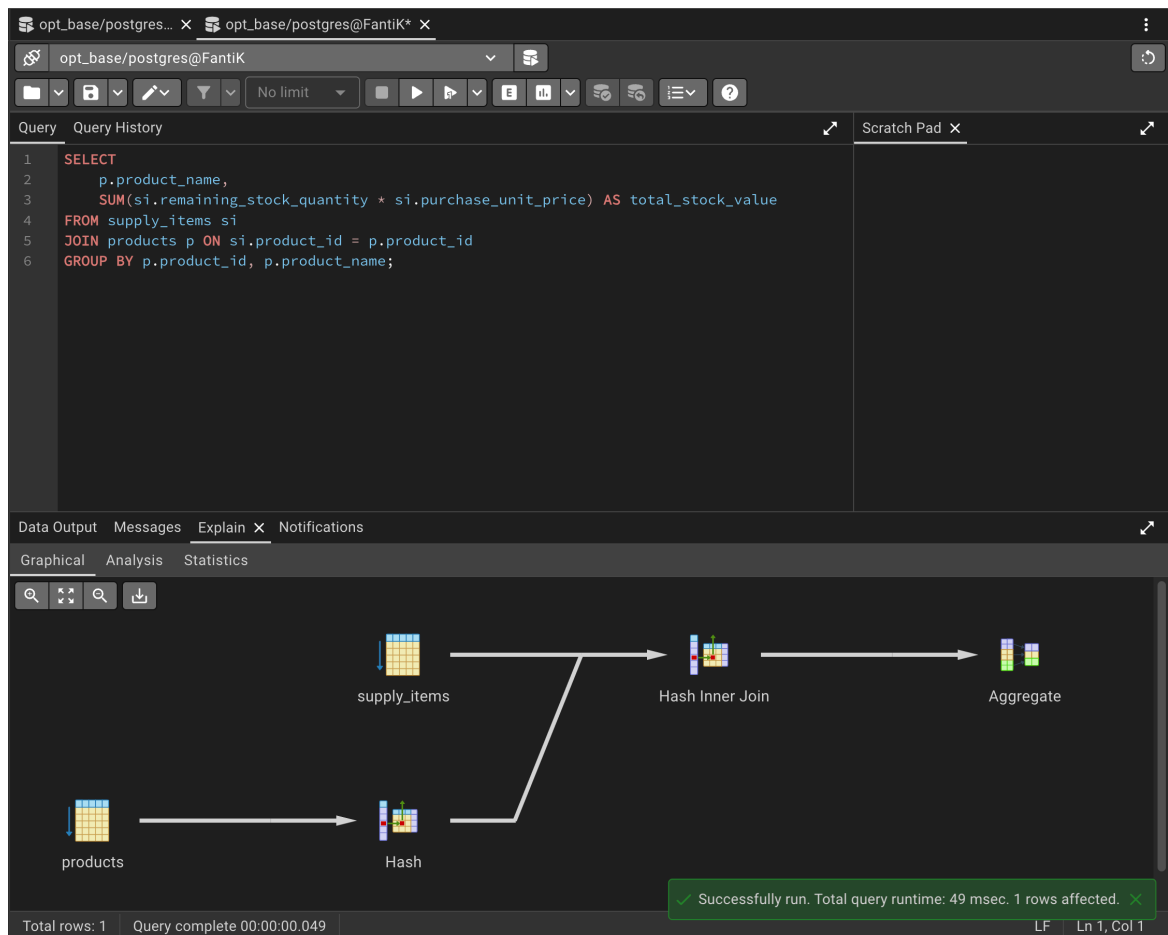


Рисунок 21 — Графический план запроса (вкладка Explain)

План выполнения показывает последовательность операций: последовательное сканирование таблицы `supply_items`, последовательное сканирование таблицы `products` с построением хеш-таблицы (Hash), их соединение через Hash Inner Join и финальную агрегацию (Aggregate), соответствующую SUM и GROUP BY в запросе.

Для просмотра активности выполненных команд использована вкладка Query History в Query Tool. На вкладке отображается список всех запросов, выполненных в текущей сессии, с указанием времени выполнения; при выборе конкретного запроса справа показываются дата и время, количество затронутых строк, продолжительность выполнения и полный текст команды.

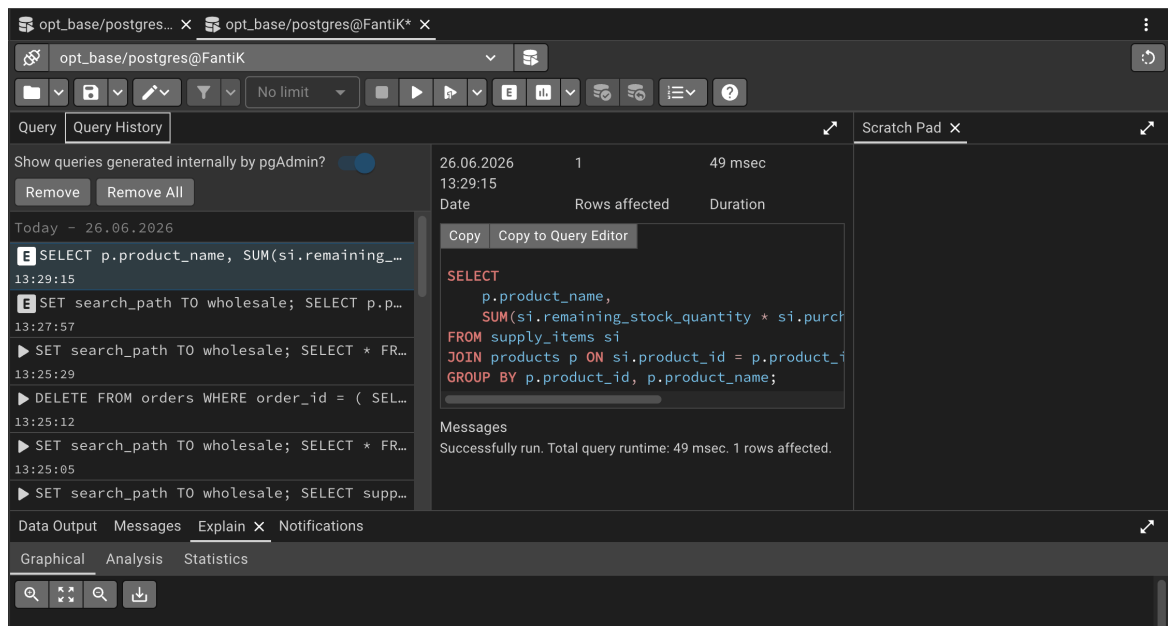


Рисунок 22 — Вкладка *Query History* с историей выполненных запросов

Создание индексов и сравнение производительности запросов

Примечание: так как таблицы базы данных содержат небольшое количество строк (от 3 до 9), планировщик PostgreSQL по умолчанию использует последовательное сканирование (Seq Scan) как наиболее быстрый способ выполнения запроса даже при наличии индекса. Для демонстрации фактического использования индекса планировщиком в плане запроса временно отключалось последовательное сканирование командой `SET enable_seqscan = OFF` — это стандартный приём для учебной демонстрации работы индексов на таблицах малого объёма.

Простой индекс

Для демонстрации использован запрос поиска товаров по коду производителя в таблице `products`:

Query

```

1 SET search_path TO wholesale;
2
3 EXPLAIN ANALYZE
4 SELECT * FROM products WHERE manufacturer_id = 1;

```

Data Output Messages Explain X Notifications

Showing rows: 1 to 6 Page No: 1 of 1

QUERY PLAN
1 Seq Scan on products (cost=0.00..11.12 rows=1 width=864) (actual time=0.056..0.057 rows=1.00 loop...
2 Filter: (manufacturer_id = 1)
3 Rows Removed by Filter: 2
4 Buffers: shared hit=1
5 Planning Time: 0.122 ms
6 Execution Time: 0.076 ms

Total rows: 6 Query complete 00:00:00.050 LF Ln 4, Col 50

Рисунок 23 — План запроса без индекса (Seq Scan), Execution Time = 0.076 ms

Создание простого индекса по столбцу manufacturer_id:

Query

```

1 CREATE INDEX idx_products_manufacturer_id ON products (manufacturer_id);

```

Data Output Messages Explain X Notifications

CREATE INDEX

Query returned successfully in 58 msec.

Рисунок 24 — Создание простого индекса idx_products_manufacturer_id

Повторное выполнение запроса с принудительным использованием индекса:

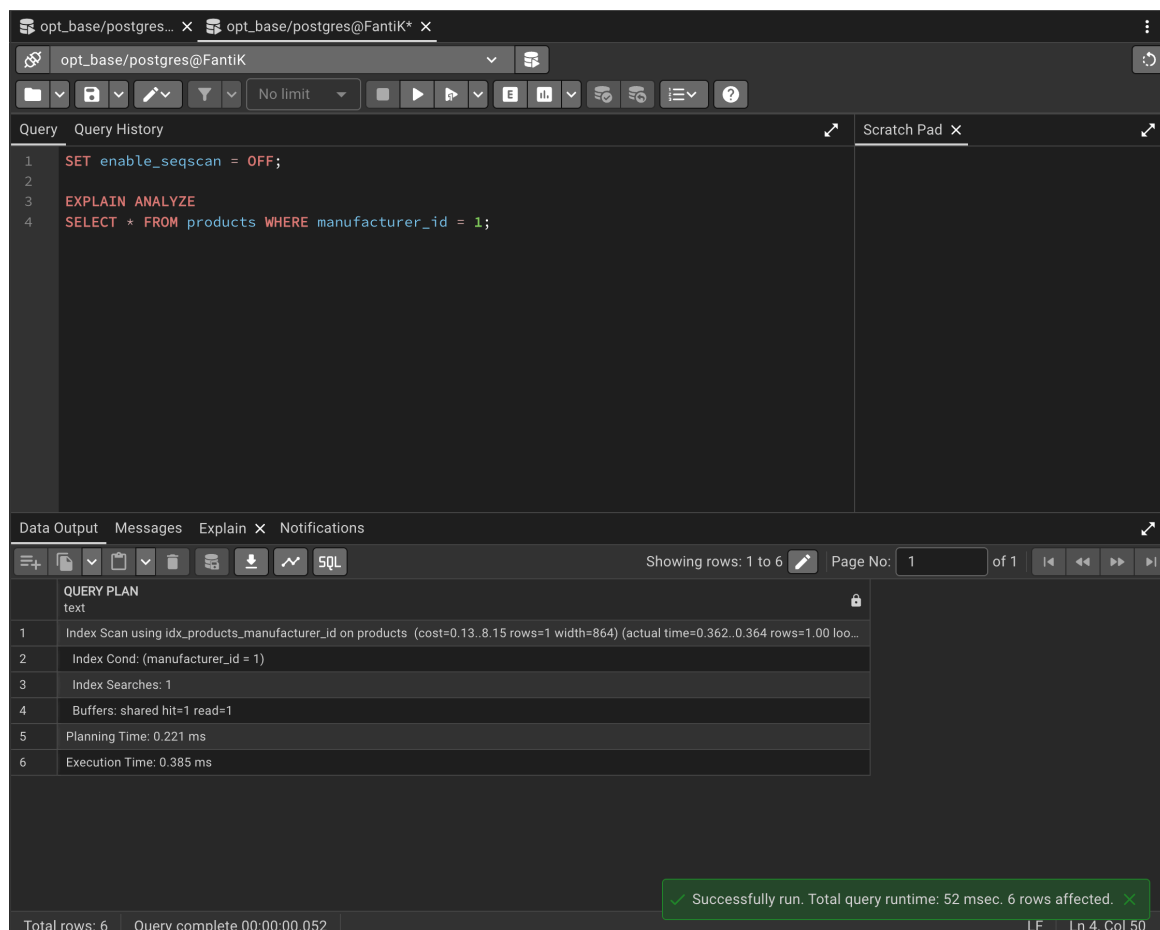


Рисунок 25 — План запроса с использованием индекса (Index Scan), Execution Time = 0.385 ms

Без индекса план запроса использует Seq Scan (полное сканирование таблицы) с временем выполнения 0.076 ms. После создания индекса и принудительного отключения последовательного сканирования план запроса использует Index Scan using idx_products_manufacturer_id с временем выполнения 0.385 ms. На данном объёме данных (3 строки в таблице products) время выполнения с индексом оказалось больше, чем без него — это ожидаемое поведение: при малом количестве строк дополнительное обращение к структуре индекса обходится дороже, чем простое сканирование всей таблицы. Преимущество индексов проявляется на таблицах с большим количеством строк (от тысяч и более), где избежать полного сканирования таблицы становится существенно выгоднее.

Составной индекс

Для демонстрации использован запрос поиска поставок по двум условиям одновременно — коду поставщика и номеру договора в таблице `supplies`:

План запроса без индекса:

The screenshot shows a PostgreSQL query editor interface. The query window contains the following SQL code:

```
1 SET search_path TO wholesale;
2
3 EXPLAIN ANALYZE
4 SELECT * FROM supplies WHERE supplier_id = 1 AND contract_number = 'SUP-2025-001';
```

The bottom panel displays the query plan for the executed query:

Step	Operation	Details
1	Seq Scan on supplies	(cost=0.00..11.50 rows=1 width=730) (actual time=0.078..0.080 rows=3.00 loop=1)
2	Filter	((supplier_id = 1) AND ((contract_number)::text = 'SUP-2025-001'::text))
3	Rows Removed by Filter	6
4	Buffers	shared hit=1
5	Planning Time	0.194 ms
6	Execution Time	0.112 ms

A green status bar at the bottom right indicates: "Successfully run. Total query runtime: 39 msec. 6 rows affected."

Рисунок 26 — План запроса без индекса (*Seq Scan*), *Execution Time* = 0.112 ms

Создание составного индекса по двум столбцам — `supplier_id` и `contract_number`:

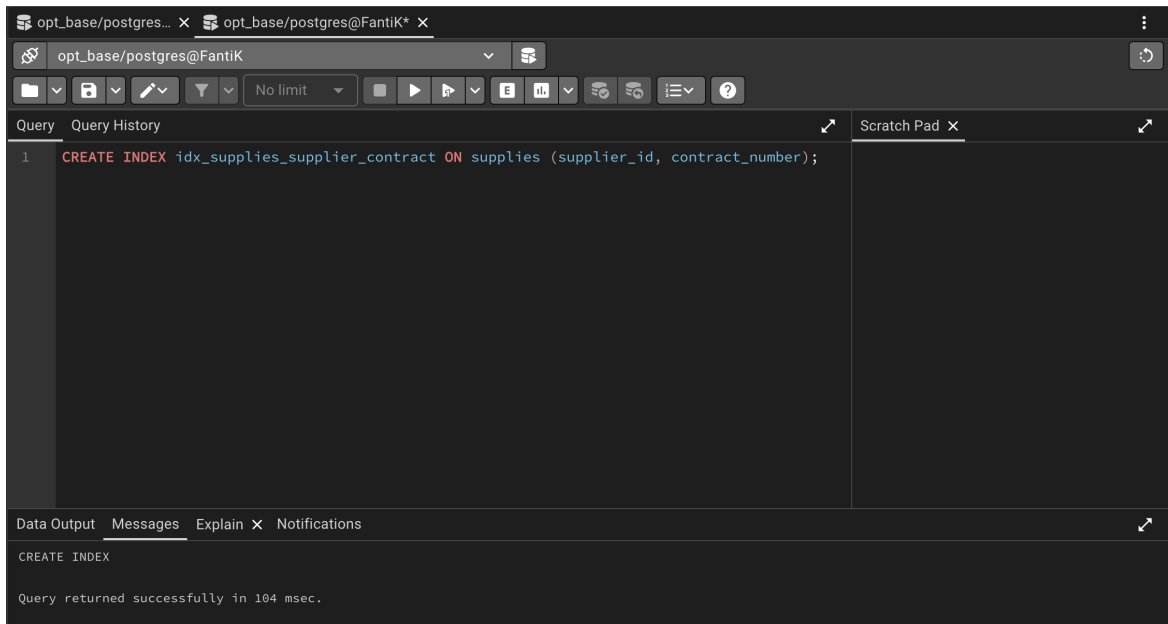


Рисунок 27 — Создание составного индекса `idx_supplies_supplier_contract`

Повторное выполнение запроса с принудительным использованием индекса:

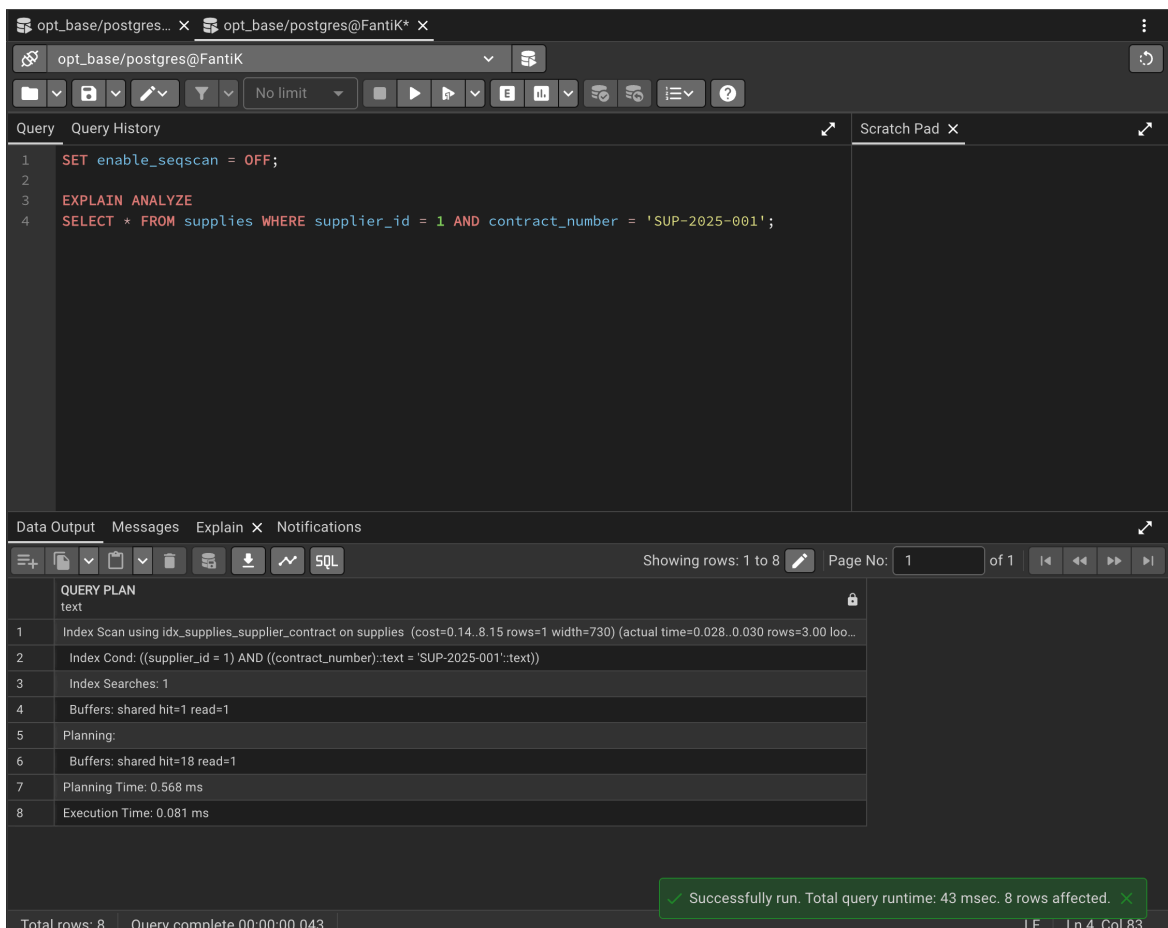


Рисунок 28 — План запроса с использованием составного индекса (*Index Scan*), *Execution Time = 0.081 ms*

Без индекса план запроса использует Seq Scan с фильтрацией по обоим условиям (`supplier_id = 1 AND contract_number = 'SUP-2025-001'`), время выполнения — 0.112 ms. После создания составного индекса и принудительного отключения последовательного сканирования план запроса использует Index Scan using `idx_supplies_supplier_contract`, при этом оба условия отображаются как единое условие индекса (Index Cond), а не как отдельный фильтр после сканирования — время выполнения составило 0.081 ms. В данном случае составной индекс показал небольшое преимущество по времени, хотя разница в десятые доли миллисекунды на таком малом объёме данных статистически малозначима.

Удаление индексов

После завершения сравнения созданные индексы были удалены, а параметр `enable_seqscan` возвращён в значение по умолчанию:

```
SET enable_seqscan = ON;  
  
DROP INDEX idx_products_manufacturer_id;  
DROP INDEX idx_supplies_supplier_contract;
```

Удаление индексов подтверждено повторным запросом к системному каталогу `pg_indexes` — после выполнения `DROP INDEX` результат запроса пуст, что подтверждает отсутствие созданных ранее индексов в схеме `wholesale`:

```
SELECT indexname FROM pg_indexes  
WHERE schemaname = 'wholesale' AND indexname LIKE 'idx_%';
```

Запрос вернул 0 строк — оба временных индекса (`idx_products_manufacturer_id` и `idx_supplies_supplier_contract`) успешно удалены из базы данных.

Выводы

В ходе выполнения лабораторной работы №4 были получены практические навыки создания запросов на выборку данных, представлений, запросов на модификацию данных с использованием подзапросов, а также работы с индексами в СУБД PostgreSQL средствами pgAdmin 4.

По индивидуальному заданию (вариант 9, БД «Оптовая база») выполнены 7 запросов на выборку данных, охватывающих агрегирующие функции, группировку, подзапросы и операции сравнения, а также 2 представления, упрощающие повторное обращение к часто используемым выборкам данных (количество заказов покупателей за период, доход базы по дням).

Составлены и выполнены 3 запроса на модификацию данных (INSERT, UPDATE, DELETE), каждый из которых использует подзапрос для определения конкретной записи или условия модификации, что подтверждено сравнением состояния таблиц до и после выполнения каждой команды.

Изучено графическое представление плана запроса на вкладке Explain и история выполненных команд на вкладке Query History. Выполнено сравнение производительности запросов без индекса и с индексом для простого индекса (по одному столбцу) и составного индекса (по двум столбцам): на используемом объёме тестовых индексы не дают заметного прироста производительности, что соответствует ожидаемому поведению планировщика PostgreSQL — на малых таблицах последовательное сканирование обходится дешевле обращения к индексу. Преимущество индексов проявляется на таблицах большого объёма. По завершении сравнения созданные индексы были удалены.